Learning a distribution

An AI image generator only ever sees a finite set of training images. What should it learn from them so it doesn't just copy them?

Oct 8, 2026 Retrieved Nov 7, 2026

You will learn

- How a network can turn random numbers into new images.
- Why far fewer random numbers than pixels can be enough to make an image.
- Why using so few numbers rules out the most common way of training a model, and what can be done about it.
- What training really produces on a small example, and how to catch a model that copies its training images.
- Where the error of a trained model comes from.

Before you start

- Random variables, densities and expectations
- What a neural network is, and training by gradient descent
- Multivariable calculus for the formal statements, which can be skipped on a first read

Most of the neural networks we meet first are trained to recognize things. We show one an image of a handwritten digit and it tells us which digit it is. Now imagine asking for the opposite. After seeing many handwritten digits, the network should draw new ones that nobody wrote. A network that does this is a *generative model*, and this course is about the mathematics that makes it work.

The figure below shows such a model at work. On the left are digits from MNIST, a standard dataset whose training set holds $60{,}000$ handwritten digits. On the right are digits drawn by a generator, our word for a network that draws images, trained on that set. Almost none of them are copies of training digits.

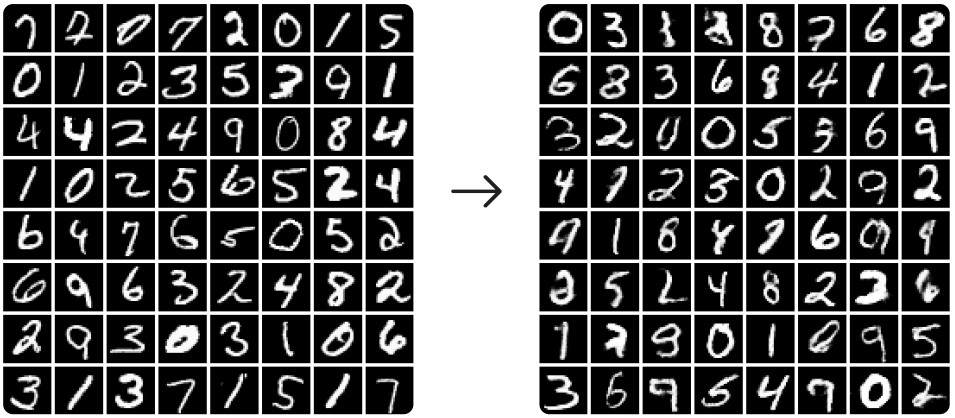


Figure 1. Left, 64 digits from the MNIST training set. Right, 64 digits drawn by a small generator trained on that set (a Wasserstein GAN, the subject of Chapter 5).

That last detail is the heart of the problem, because copying would have been easy. A model that stores the $60{,}000$ digits and returns one at random matches the training set perfectly, and we call it the *copying model*. It is useless, because it can only show digits we already have. What we want is harder to say. A generator should learn the process that produced the digits, and the training set is the only evidence about that process. This chapter is about what that asks of a network.

The learning problem

Before writing anything formal, we want an example we can look at. Images are a poor choice, since each one is a point in a space with hundreds of dimensions where no distribution can be plotted. So this chapter works with a much smaller example in which every sample is visible, namely 12 points drawn at random on the unit circle. Think of the 12 points as our training set and of the circle as the process that produced them, hidden from the model. The copying model returns one of the 12 points. The model we want produces points anywhere on the circle, since that is where the 12 came from.

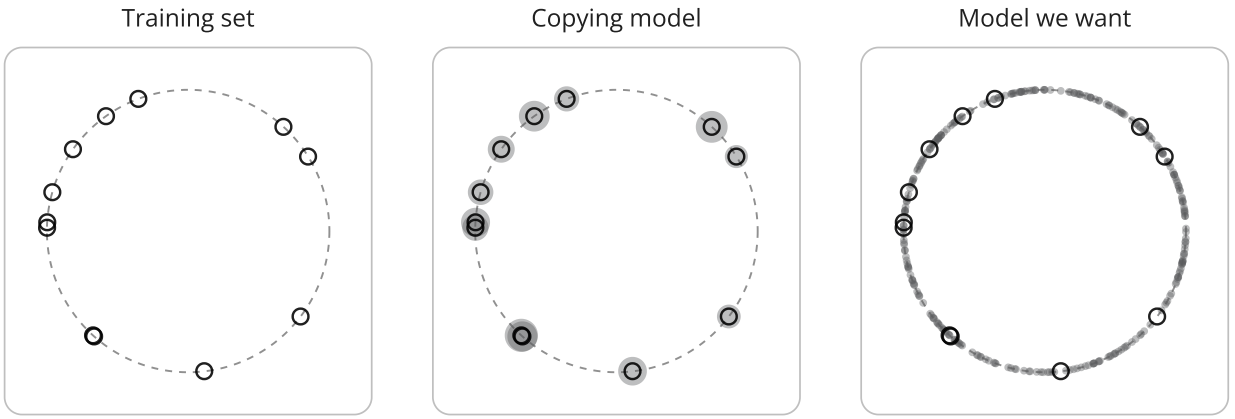


Figure 2. Left, the 12 training points (open circles) on the unit circle (dashed), which the model never sees. Middle, 300 samples of the copying model. Each sample is one of the 12 points, and the grey disc around a point grows with the number of samples that landed on it. Right, 300 samples of the model we want, spread over the whole circle.

Now the same situation in symbols. We observe n samples $X_1, \dots, X_n$ drawn independently from an unknown probability distribution $\mathbb{P}^*$ on $\mathbb{R}^d$. On the circle, $d = 2$, $n = 12$ and $\mathbb{P}^*$ is the uniform distribution on the unit circle. For images, each sample is a vector with one coordinate per pixel and colour channel, and d is the number of coordinates. A grey MNIST digit has $d = 28 \times 28 = 784$ and a CIFAR-10 photo has $d = 32 \times 32 \times 3 = 3072$. All we know about $\mathbb{P}^*$ comes from the samples. The plainest summary of them puts equal weight on each one, and it has a name.

Definition 1 (Empirical distribution)**.** The *empirical distribution* of $X_1, \dots, X_n$ is

$$\mathbb{P}_n = \frac{1}{n} \sum_{i=1}^{n} \delta_{X_i},$$

where δ_x is the point mass at x. Drawing from $\mathbb{P}_n$ means picking one of the training images uniformly at random.

Sampling from $\mathbb{P}_n$ is exactly what the copying model does, so the copying model now has a formula. On the circle, $\mathbb{P}_n$ puts mass $1/12$ on each training point, while $\mathbb{P}^*$ spreads its mass evenly over the whole circle. What we want is a distribution $\widehat{\mathbb{P}}$ built from the samples that is close to $\mathbb{P}^*$ and that we can sample from, since new images are what we are after. To get there we first need a set of candidate networks, since a network has to describe a distribution before we can ask how far it is from $\mathbb{P}^*$.

Three ways to describe a distribution

A network takes numbers in and gives numbers out, so it is not obvious how it can describe a distribution on $\mathbb{R}^d$. The question is what those output numbers should be. Three answers are in common use.

The most direct answer asks the network for a *density* $p_\theta(x)$, a function that says how likely each image is. This is harder than it sounds. A density must be non-negative and integrate to 1, but a flexible neural network meets neither condition on its own. Forcing a network to output a normalized density restricts the architecture heavily.

Another answer drops the normalization. The network outputs a function that is only proportional to the density, or the gradient of its logarithm in x. That gradient is the *score*, and a one-dimensional case shows why it is convenient. For the standard Gaussian, $\log p(x) = -x^2/2 - \frac{1}{2}\log(2\pi)$, so the score is $-x$. At $x = 2$ it equals -2 and points back toward the centre, and the further we go the harder it pulls. In general the score points toward images of higher density. The normalizing constant (here the term with $\log(2\pi)$) vanishes in the gradient, so the network has no normalization to enforce. The network only has to output a vector of the same size as its input, so the usual image architectures can be trained to estimate a score as they are.

This freedom has two costs. The model no longer gives probabilities. Drawing a sample also takes many steps that follow the score, because the score only says which way to move. The box below gives the formulas for both answers.

Going further

Writing down a density

An **autoregressive model** orders the d coordinates (one colour channel of one pixel each) and writes the distribution as a product of one-dimensional conditionals,

$$p_\theta(x) = \prod_{i=1}^{d} p_\theta(x_i \mid x_1, \dots, x_{i-1}).$$

Pixel values are discrete, so each factor is a probability table over the 256 intensity levels, and the product sums to 1. Spreading each level uniformly over an interval of width $1/256$ turns it into a density on $\mathbb{R}^d$ (Theis, Oord, and Bethge 2016). The negative log-likelihood is a sum of d classification (cross-entropy) losses, and sampling takes d network calls, one coordinate at a time (Oord, Kalchbrenner, and Kavukcuoglu 2016).

A **normalizing flow** uses a map $T_\theta : \mathbb{R}^d \to \mathbb{R}^d$ that is invertible and differentiable with a differentiable inverse, and a latent distribution γ with a density $p(z)$. If $Z \sim \gamma$ and $X = T_\theta(Z)$, the change of variables gives

$$\log p_\theta(x) = \log p\big(T_\theta^{-1}(x)\big) + \log \left| \det \frac{\partial T_\theta^{-1}}{\partial x}(x) \right|.$$

The likelihood is computable exactly when the inverse and the determinant are cheap, which constrains the architecture (Rezende and Mohamed 2015; Dinh, Sohl-Dickstein, and Bengio 2017). The noise of a flow has as many coordinates as the image, so Proposition 6 below does not apply, and the change of variables gives the flow a density.

An **energy-based model** leaves the density unnormalized. It takes any network $E_\theta : \mathbb{R}^d \to \mathbb{R}$ and sets $p_\theta(x) \propto e^{-E_\theta(x)}$, so the architecture is free. The cost moves to the normalizing constant $\int e^{-E_\theta(x)}\, dx$, an integral over all of $\mathbb{R}^d$ that has no closed form for a general network. In high dimension it can only be estimated by sampling from the model. The *score* $\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$ does not contain this constant, and a network can be trained to estimate the score of the data directly from samples (Hyvärinen 2005). The diffusion models of Chapter 7 learn the score of the data blurred by noise at many levels (Vincent 2011; Song and Ermon 2019).

The last answer goes straight to what we are after. It never writes the density down. Instead it says how to produce a sample: draw random noise from a simple distribution, then pass the noise through a network that turns it into an image.

Generators and the pushforward

Both steps of this recipe get a name. The noise is a random vector Z with a fixed distribution γ on $\mathbb{R}^k$, usually the standard Gaussian $\mathcal{N}(0, I_k)$, which is easy to sample and has no parameters. We call Z the *latent variable*, since it never appears in the data. The network that turns Z into an image is the *generator* $g : \mathbb{R}^k \to \mathbb{R}^d$, a neural network with weights θ. A definition like this stays abstract until we see a generator written out. The circle lets us write one that produces $\mathbb{P}^*$ exactly.

Example 2 (A generator for the circle)**.** Take $k = 1$, let $Z \sim \mathcal{N}(0, 1)$, and define g by

$$g(z) = \big(\cos 2\pi\Phi(z),\, \sin 2\pi\Phi(z)\big),$$

where Φ is the distribution function of the standard Gaussian. The random variable $\Phi(Z)$ is uniform on $[0, 1]$, so the angle $2\pi\Phi(Z)$ is uniform on $[0, 2\pi)$ and $g(Z)$ is uniform on the unit circle. Each draw of Z gives a new point of the circle. Taken over all draws, the distribution of $g(Z)$ is exactly $\mathbb{P}^*$.

The step that does the work is that $\Phi(Z)$ is uniform. For y in $(0, 1)$, the probability that $\Phi(Z) \le y$ is the probability that $Z \le \Phi^{-1}(y)$, which is $\Phi(\Phi^{-1}(y)) = y$.

Notice what just happened. The generator turned a Gaussian on a line into a uniform distribution on a circle. We found that distribution without writing a density for it. The distribution of $g(Z)$ is what training will adjust, so it deserves a name. Its definition answers one question, how to find the probability of a set A of images. We go back to the latent points that g sends into A and ask how likely those are.

Definition 3 (Pushforward)**.** Let $Z \sim \gamma$ on $\mathbb{R}^k$ and let $g : \mathbb{R}^k \to \mathbb{R}^d$ be measurable. The *pushforward* of γ by g is the distribution of $g(Z)$, written $g_{\#}\gamma$:

$$(g_{\#}\gamma)(A) = \gamma\big(g^{-1}(A)\big) \quad \text{for every measurable set } A \subset \mathbb{R}^d.$$

We write $\mathbb{P}_g = g_{\#}\gamma$.

Here $g^{-1}(A) = \{z \in \mathbb{R}^k : g(z) \in A\}$ is that set of latent points, so $\gamma\big(g^{-1}(A)\big)$ is the probability that Z lands in it. The set is defined whether or not g is invertible. *Measurable* is a technical condition satisfied by all continuous functions and all sets in this series. For the circle generator, $\mathbb{P}_g = \mathbb{P}^*$. Applied to an arc, the definition says that its probability is the probability that $2\pi\Phi(Z)$ falls in the corresponding range of angles.

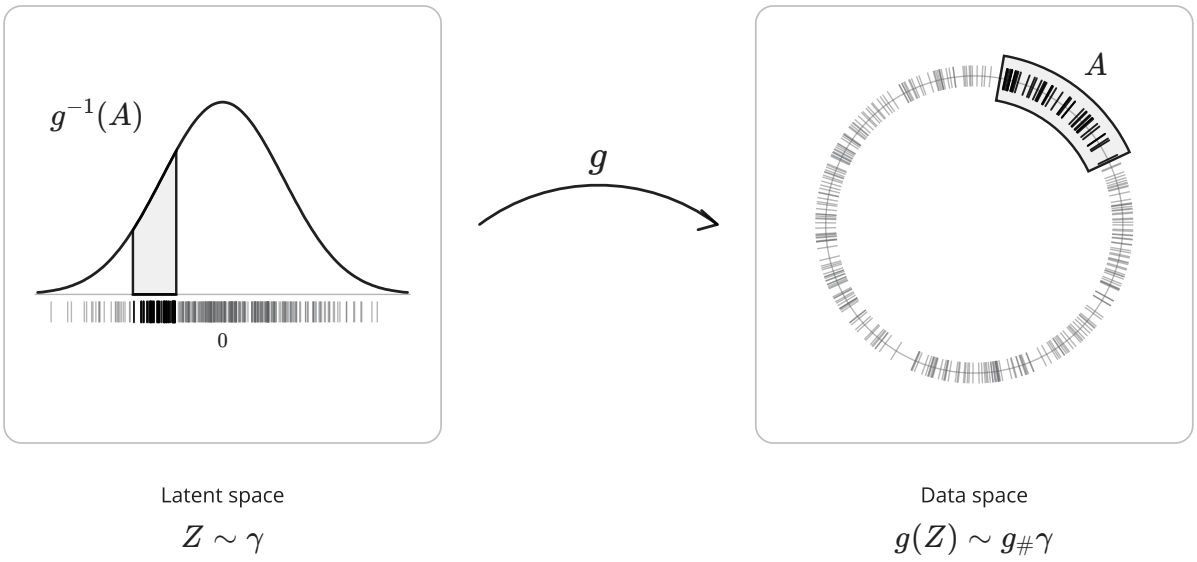


Figure 3. The circle generator of Example 2 pushes the Gaussian γ on the line forward to the uniform distribution on the circle. Each tick is a sample $Z \sim \gamma$ on the left and its image $g(Z)$ on the right. The samples that land in the region A (dark ticks) are exactly those whose Z falls in the interval $g^{-1}(A)$, so both carry the same probability $(g_{\#}\gamma)(A) = \gamma(g^{-1}(A))$, the shaded area under the density.

In practice the pushforward is easy to use. To sample, we draw $Z \sim \gamma$ and compute $g(Z)$. Each new draw of Z gives a new image while g stays fixed. Changing the weights θ changes g, and with it the distribution of $g(Z)$. The generators obtained for all values of θ form the set of candidates we asked for. We call this set the *model class* $\mathcal{G}$. Learning means choosing $\hat{g} \in \mathcal{G}$ so that $\widehat{\mathbb{P}} = \mathbb{P}_{\hat{g}}$ is close to $\mathbb{P}^*$.

One number in this recipe is still free, the latent dimension k. The circle generator needs a single latent number to produce a curve in the plane. One might expect a 784-pixel digit to need 784 random numbers. Far fewer are enough, because images are far from arbitrary points of $\mathbb{R}^d$.

Images lie near a manifold

To see why, compare a digit with an image drawn at random. An image whose 784 pixels are drawn independently and uniformly at random looks like noise. Natural images obey regularities that such random images almost never satisfy. Neighbouring pixels usually have similar values, and the strokes of a digit form continuous lines of roughly constant width. Each of these regularities is a strong constraint. Together they confine natural images to a tiny part of $\mathbb{R}^d$.

Within that part, natural images can still change, but only along few directions. If we change the slant of a handwritten 4 or the thickness of its strokes, every intermediate image is still a plausible 4. Most other directions of $\mathbb{R}^d$, such as adding independent noise to every pixel, quickly leave the set of plausible digits. We have met this shape already. The circle sits in the plane, yet a single angle locates a point on it. Natural images lie near a surface of the same kind, thin and of low dimension inside $\mathbb{R}^d$, which mathematicians call a manifold.

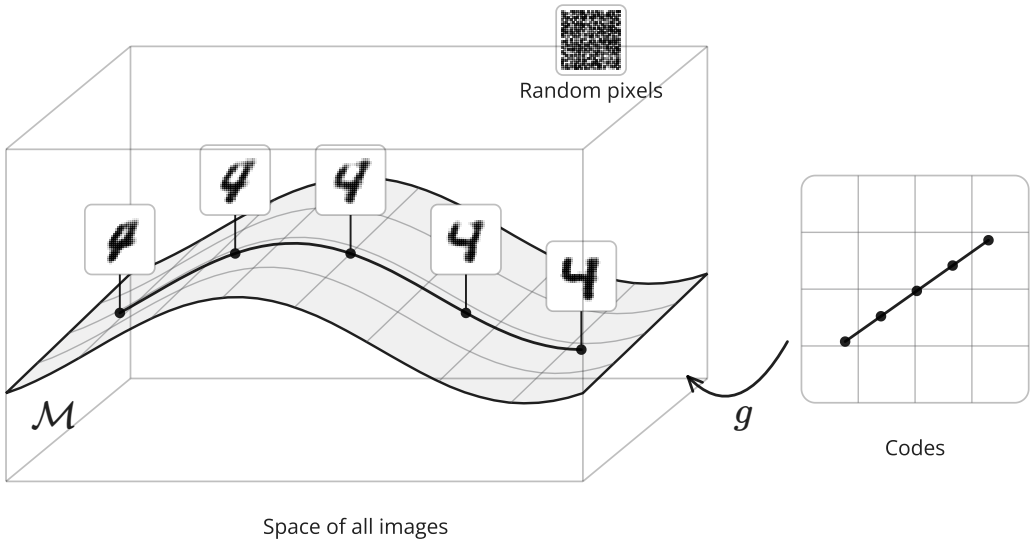


Figure 4. A path on the manifold of digits. A generator maps a straight segment of codes (right) to a curved path on the manifold $\mathcal{M}$ of plausible digits (left), so every image along the path is a plausible digit. An image with random pixels lies far from the manifold.

To use this idea we need to say what a surface of low dimension is. The requirement is local. Near any point of the surface, a few numbers should be enough to say where we are, and changing those numbers should move us smoothly along it.

Definition 4 (Manifold)**.** A set $\mathcal{M} \subset \mathbb{R}^d$ is an *m-dimensional manifold* if every point of $\mathcal{M}$ has a neighbourhood in $\mathcal{M}$ (the part of $\mathcal{M}$ inside a small ball around the point) described by m coordinates. This means there is a continuously differentiable invertible map φ from an open subset of $\mathbb{R}^m$ onto the neighbourhood, with a continuous inverse and a derivative of rank m at every point.

The conditions on φ say that the coordinates vary smoothly and that no direction is lost. The circle is a 1-dimensional manifold, with the angle as coordinate. The surface of a sphere is 2-dimensional, with latitude and longitude as coordinates. No single pair of coordinates covers the whole sphere, since longitude breaks down at the poles and jumps along one meridian, which is why the definition only asks for coordinates near each point. In both cases the manifold sits in a larger space, but moving along it takes fewer coordinates than the space has.

With this word in hand, the claim about images takes one line.

> **Assumption 5** (Manifold hypothesis)**.** The distribution $\mathbb{P}^*$ of natural images concentrates near a manifold $\mathcal{M}$ whose dimension m is much smaller than d.

By *concentrates near* we mean that most of the probability lies within a small distance of $\mathcal{M}$. The manifold hypothesis is an assumption about data, so it has to be tested on data. Measurements on real datasets support it. The *intrinsic dimension* of a dataset is an estimate of m. Estimators rely on the fact that in dimension m the number of data points within distance r

of a point grows like r^m. Pope et al. (2021) report 7 to 13 for MNIST ($d = 784$), 13 to 26 for CIFAR-10 ($d = 3072$) and 26 to 43 for ImageNet ($d = 224 \times 224 \times 3 = 150{,}528$).[1] In every case m is a few dozen at most, while d runs from hundreds to hundreds of thousands.

This is what makes a small latent dimension possible. With $k \geq m$ latent dimensions, $g(z)$ can move along $\mathcal{M}$ as z moves in $\mathbb{R}^k$, as the circle generator does with $k = m = 1$. The match is not exact. The data lie *near* the manifold, but a generator with $k < d$ puts all its mass *on* a surface of dimension at most k. The next section shows what this difference costs.

Generators without a density

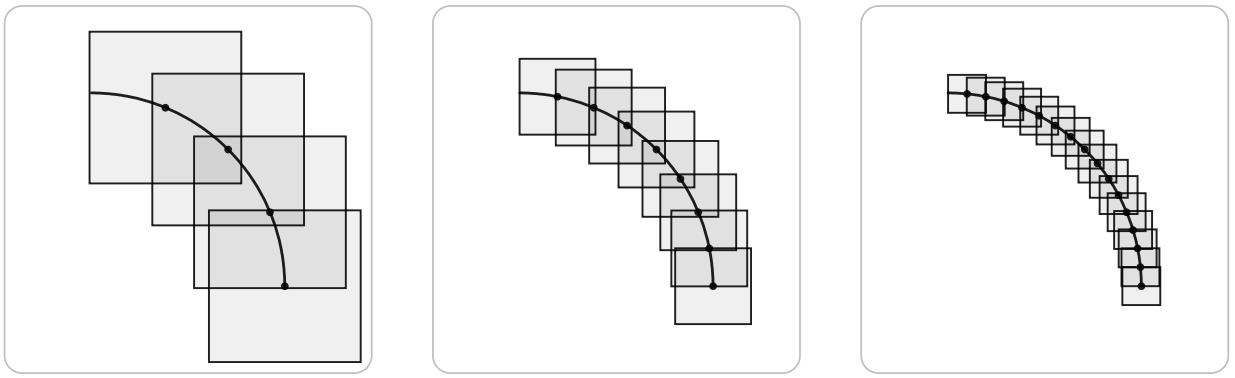
Putting all the mass on a thin surface looks harmless, but it has a consequence for training. The most common way to train a model, maximum likelihood, needs a density. A distribution $\mathbb{P}$ on $\mathbb{R}^d$ has a *density* p when the probability of every region A is the volume integral $\mathbb{P}(A) = \int_A p(x)\,dx$, where p is a function with values in $[0, +\infty)$. Maximum likelihood raises $p_\theta(X_i)$ at every training image X_i, so without a density it has nothing to raise. The circle generator has no density, since it puts all its probability on a curve and a curve has no area. The cause is not specific to the circle. It lies in a property that the usual neural networks share, and that rules out a density whenever $k < d$.

The idea is that a network cannot spread a low-dimensional input over a higher-dimensional space, because it can only stretch distances by a bounded factor. A map g is *Lipschitz* with constant L if $\|g(z) - g(z')\| \leq L\|z - z'\|$ for all z, z', so two points never end up more than L times farther apart than they started. Networks built from linear layers and the usual activations (ReLU, SiLU, tanh) are Lipschitz. The bound does real work here. Without one, a continuous curve can pass through every point of a whole square, as the space-filling curves of Peano and Hilbert do.

We go through the argument slowly. The simplest case is $k = 1$ and $d = 2$, a generator that maps a segment of length 1 to a curve in the plane. The plan is to cover the curve with small squares and show that their total area can be made as small as we like. If we split the segment into N pieces of length $1/N$, each piece becomes a stretch of curve whose points all lie within L/N of its starting point, because the network stretches distances by at most L. That stretch of curve fits inside a square of side $2L/N$ centred on the starting point. Together the N squares cover the whole curve, with total area

$$N \times \left(\frac{2L}{N}\right)^2 = \frac{4L^2}{N}.$$

A finer split means more squares, but their number grows like N while the area of each one falls like $1/N^2$, so the total shrinks. Doubling N halves this area, so the curve can be covered by squares of total area as small as we like. So the curve has no area.



$N = 4$
total area 2.47

$N = 8$
total area 1.23

$N = 16$
total area 0.62

Figure 5. The curve is a quarter of the unit circle. It is the image of $[0, 1]$ under $g(t) = (\cos \frac{\pi t}{2}, \sin \frac{\pi t}{2})$, with $L = \pi/2$. Each panel covers it with N squares of side $2L/N$, centred on the starting points of the pieces (dots). Each doubling of N halves the total area π^2/N of the squares while the curve stays covered. The curve therefore has area zero.

The same count works in any dimension. In general there are N^k pieces, each inside a box of volume proportional to N^{-d}, since each of its d sides shrinks like $1/N$. Together they cover the image with a total volume proportional to N^{k-d}, which goes to zero whenever $k < d$. With $k = 1$ and $d = 2$ the exponent is -1, the halving we just saw. The number of pieces grows with the latent dimension and the volume of each box shrinks with the data dimension, so $k < d$ is what makes the total vanish. For $k \geq d$ the count says nothing. A set that can be covered by an arbitrarily small total volume has *volume zero* (Lebesgue measure zero), like a curve in the plane.

The result only needs the Lipschitz bound on each bounded region, a weaker condition called *locally Lipschitz*. We can now state it in full.

> **Proposition 6** (No density with fewer latent dimensions than pixels). *Let $g : \mathbb{R}^k \to \mathbb{R}^d$ be locally Lipschitz with $k < d$. Then the image $g(\mathbb{R}^k)$ has volume zero in $\mathbb{R}^d$. Consequently, for every distribution γ on $\mathbb{R}^k$ the pushforward $\mathbb{P}_g = g_{\#}\gamma$ has no density on $\mathbb{R}^d$.*

Volume zero rules out a density for a short reason. A density would give the set $g(\mathbb{R}^k)$ probability 0, while $\mathbb{P}_g$ gives it probability 1. No choice of p escapes this, even one with huge values on the curve, because the integral of a function over a set of volume zero is 0. The Dirac delta of physics is a common source of confusion here, because it looks like such a function. It is a point mass, which is a distribution without a density. Arjovsky and Bottou (2017) prove the same conclusion for networks built from affine maps and pointwise nonlinearities (their Lemma 1).

Proof.

Cover $\mathbb{R}^k$ by countably many closed cubes of side 1. On one such cube C the map g is Lipschitz with some constant L, because a locally Lipschitz map is Lipschitz on every compact convex set. Split C into N^k small cubes of side $1/N$. Each has diameter $\sqrt{k}/N$, so its image has diameter at

most $L\sqrt{k}/N$ and fits in a ball of radius $L\sqrt{k}/N$. That ball has volume $\omega_d (L\sqrt{k})^d N^{-d}$, where ω_d is the volume of the unit ball of $\mathbb{R}^d$. The image $g(C)$ is therefore covered by balls of total volume at most

$$N^k \cdot \omega_d (L\sqrt{k})^d N^{-d} = \omega_d (L\sqrt{k})^d \, N^{k-d},$$

which tends to 0 as $N \to \infty$ because $k < d$. So $g(C)$ has volume zero. Each $g(C)$ is compact as the continuous image of a compact set, so $g(\mathbb{R}^k)$ is a countable union of compact sets of volume zero. It is measurable and has volume zero too. Finally $\mathbb{P}_g(g(\mathbb{R}^k)) = 1$, while a density would give it probability 0. □

This settles what we said about the circle generator. Its derivative has norm $2\pi\Phi'(z) \le \sqrt{2\pi}$, since $\Phi'(z) \le 1/\sqrt{2\pi}$. A bound on the derivative is a bound on the stretching, so the generator is Lipschitz with constant $\sqrt{2\pi}$. It has $k = 1 < d = 2$, so the proposition applies.

The same covering argument also tells us how small the latent dimension may be, the question left open two sections ago. For this we need a different notion of size, because volume in $\mathbb{R}^d$ cannot tell a long curve from a short one, as both have volume zero. We use m-dimensional volume, which is length for $m = 1$ and area for $m = 2$ (its formal name is the Hausdorff measure). Measured this way, a locally Lipschitz generator with $k < m$ cannot cover an m-dimensional manifold. The count is the same as before. There are N^k pieces, each of m-dimensional volume proportional to N^{-m}, for the same reason as the boxes above. The image therefore has m-dimensional volume proportional to N^{k-m}, which goes to zero when $k < m$. A manifold of dimension m has positive m-dimensional volume, as a curve has positive length, so the image cannot cover it. A generator needs at least m latent dimensions for its image to cover the manifold. With fewer it can still come close to $\mathbb{P}^*$ in the Wasserstein distance of Chapter 2, which measures how far probability has to travel, as a curve can wind through a disk and pass near all of its points. Such a curve is long, so its Lipschitz constant is large.

So a generator with $k < d$ has no density, which leaves maximum likelihood nothing to maximize. The VAE of Chapter 3 fixes this by adding a little Gaussian noise to the output, $X = g(Z) + \sigma_{\text{dec}}\,\varepsilon$, where $\varepsilon \sim \mathcal{N}(0, I_d)$ is drawn independently of Z and the number $\sigma_{\text{dec}} > 0$ sets the size of the noise. Each point of the curve then becomes a small fuzzy blob, so every point of the plane gets a finite, positive density

$$p_\theta(x) = \int \mathcal{N}\big(x;\, g(z),\, \sigma_{\text{dec}}^2 I_d\big)\, p(z)\, dz,$$

where $\mathcal{N}(x; \mu, \sigma^2 I_d)$ is the Gaussian density at x and $p(z)$ is the density of γ. The integral adds up one small blob for every latent point, weighted by how likely that point is. This density is large near the curve and fades with the distance to it. The noisy model is still a pushforward. It pushes the pair (Z, ε) forward through $(z, \varepsilon) \mapsto g(z) + \sigma_{\text{dec}}\,\varepsilon$, whose latent dimension $k + d$ is not smaller than d, so Proposition 6 does not apply. When the curve passes beside a training point, $p_\theta(X_i)$ is small but not zero. It grows as the curve moves toward the point, so maximizing the

likelihood pulls the generator toward the data. Diffusion models in Chapter 7 add noise at every step for the same reason. Chapter 2 takes the other route and looks for distances that make sense without any density.

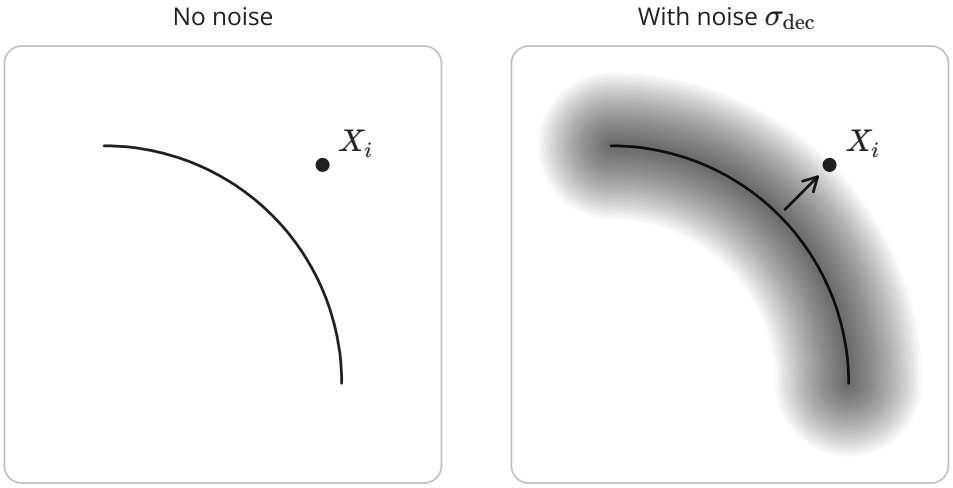


Figure 6. Left, a generator without noise puts all its mass on the curve, so a training point X_i beside it has density zero however close the curve comes. Right, Gaussian noise spreads the curve into a band and every point gets a positive density. Moving the curve toward X_i raises its likelihood.

The main runs of the next section use $k = d = 2$, where the proposition does not apply and a generator can spread its mass over an area of the plane. The section ends by repeating them with $k = 1$, where it applies.

What training produces

The sections so far say what a generator can represent. They say nothing about what training will produce. The circle generator is no help here, since it was written down from $\mathbb{P}^*$, which training never sees. To find out, we train generators on a few points of the circle and look at every sample. The question is whether they learn the circle or the points.

Each generator is a network with two hidden layers of the same width and SiLU activations, mapping $\mathbb{R}^2$ to $\mathbb{R}^2$. The width grows from 4 to $1{,}024$ by factors of 4. The text follows widths 16 and $1{,}024$, while the Lab box at the end of this section lists all five.

Training needs a number to minimize. Here it is the maximum mean discrepancy (MMD) between the samples and the training points. MMD compares two clouds of points through a similarity kernel, a function κ of two points that is largest when they coincide and falls to nearly zero when they are far apart. It averages this similarity over pairs within each cloud and over pairs across the two clouds. The MMD is small when the two kinds of average agree.

The smallest case shows how this behaves. Put one point x in the first cloud and one point y in the second. Then $\mathrm{MMD}^2 = \kappa(x, x) + \kappa(y, y) - 2\kappa(x, y)$. The two terms within the clouds are fixed at the largest value of the similarity. When $x = y$ the term across the clouds cancels them and the result is 0. As y moves away that term fades, so the result rises toward twice the largest value. Larger clouds use the same three averages. Their formula is in the Going further box

"Restricted distances with fast rates and blind spots" of Chapter 2. With the Gaussian kernels used here, MMD amounts to blurring each cloud into a smooth density and measuring how much the two blurred densities differ. The losses below report its square, MMD^2.

I chose the MMD because it needs neither a density nor a second network trained alongside the generator. The first means it can train a generator with $k = 1$. The second means that the only training that can go wrong is the generator's own.

Every generator trains for $100{,}000$ steps. Every setting is repeated three times, each time with its own training points drawn from the circle, so a result that holds in all three does not depend on one lucky draw.

Going further

Why a generator cannot copy exactly

A set is *connected* if it cannot be split into two pieces that are separated from each other. The space $\mathbb{R}^k$ is connected, and so is its image by a continuous map. A Gaussian gives positive probability to every open set, so $\mathbb{P}_g$ puts mass near every point of the connected set $g(\mathbb{R}^k)$. With $n \geq 2$ distinct training points, $\mathbb{P}_n$ sits on n separated points, which a continuous generator fed with Gaussian noise cannot produce exactly. It can still come arbitrarily close in the Wasserstein distance of Chapter 2, by sending almost all of its mass near the training points and keeping thin threads in between.

A low training objective alone cannot tell us much, because the copying model is exactly what scores best on it. So we judge each generator on $2{,}000$ of its samples with three numbers.

- The *training loss* is the objective itself, the MMD between the samples and the training points.
- The *test loss* is the same MMD, measured against $2{,}000$ fresh points from the circle that the generator never saw. It estimates how far the generator is from $\mathbb{P}^*$. Fresh points of the circle score 0.002 themselves, because $2{,}000$ points are only a sample.
- *Copies* is the share of samples that land very close to a training point. Picture a small disc around each training point, with a radius chosen so that a fresh point of the circle falls in one of the discs only 1% of the time. A sample inside a disc counts as a copy, so fresh points score about 1% by construction.

All three need only a held-out set, so later chapters report them on real images too. For the MNIST generator of the opening figure the share of copies is 0.20%, which is why we could say that almost none of its digits are copies. Real test images score 1.15%, so the generator sits below them. That is a warning sign of its own. Its digits sit slightly farther from the training images than real digits do, which means they lie slightly off the data.

Before looking at the table, it helps to say what success would look like. A generator that has learned the circle behaves like fresh points of the circle, so its test loss should be close to 0.002 and its copies close to 1%. Its training loss should be about the same as that of the true circle,

the column *Truth* below. A generator that has learned the points should beat the truth on training loss, with a test loss near *Truth* and many more than 1% copies.

Width	Points	Training loss	Truth	Test loss	Copies
16	12	0.054–0.106	0.351–0.600	0.256–0.518	3.6–6.6%
1024	12	0.010–0.015	0.351–0.600	0.277–0.538	11.4–25.1%
16	200	0.005–0.007	0.016–0.025	0.011–0.032	0%
1024	200	0.004–0.007	0.016–0.025	0.016–0.029	0–0.1%

Circle generators of widths 16 and 1024 after 100,000 steps. Each range gives the smallest and largest value over the three draws of the training points. *Truth* is the training loss of 2,000 points of the true circle against the same training points.

Start with the training loss. Every generator trained on 12 points scores better on its training objective than the true circle does. A perfect generator draws its samples from $\mathbb{P}^*$ itself, yet on the same points its training loss is at least 0.35. Every trained generator, of every width, scores at most 0.22. Twelve points cover little of the circle, so a model that hugs them beats the truth on a score that only sees them.

Before running the experiment, I expected a wider generator to trace the circle more closely, since it can bend more freely. It traced the training points more closely. Trained on the same 12 points, the generator of width $1{,}024$ puts most of its samples on the training points and on the straight chords between neighbouring points. The narrow one follows the same chords more loosely and scatters more samples inside the circle. In each of the three draws the wider generator copies more. Neither is close to the circle. In every draw the test loss of both widths stays above 0.25, far from the 0.002 of fresh points.

Now compare this with what we expected. On 12 points the copies are well above the 1% of fresh points, and the test loss is close to *Truth*. Together with the training loss, all three numbers say the same thing. These generators learned the points and the chords between them. They did not learn the circle.

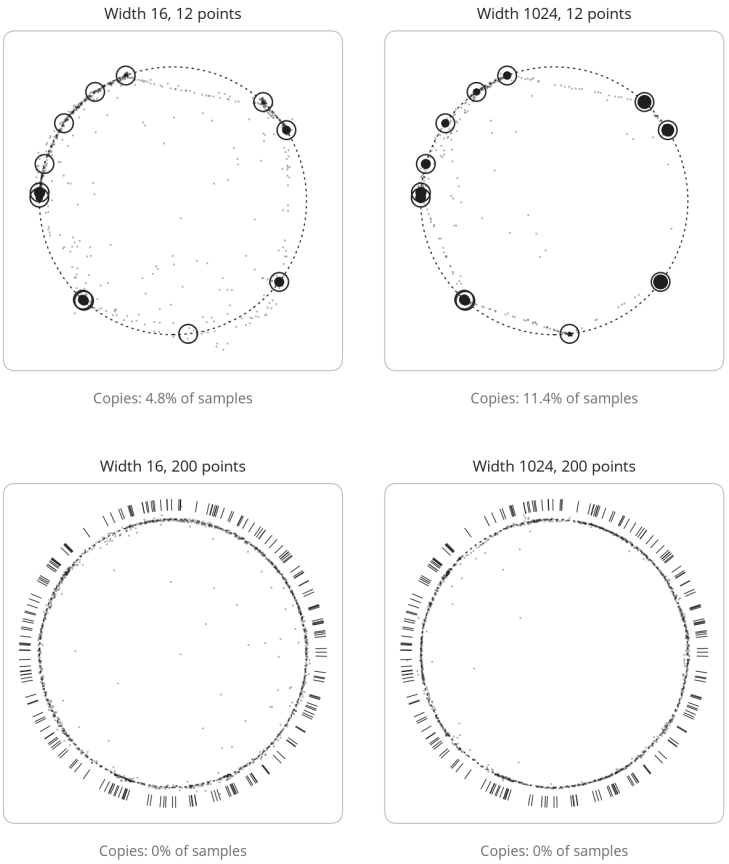


Figure 7. Samples of generators of width 16 and 1024 for one draw of the training points. The buttons under the panels switch between 12 training points (open circles) and 200 (short ticks outside the circle). Grey dots are 1,200 of the 2,000 samples. The dashed line is the true circle. Copies lie too close to their training point to be told apart from it, so the dark disc on each training point has an area proportional to the number of copies next to it. The share of copies among all 2,000 samples is given under each panel. *The online version of this figure is interactive.*

By training loss, the wide generator looks better, since it reaches the lower training loss in each draw. It is also the generator that comes closer to copying. The reason lies in the objective. It compares the generator with $\mathbb{P}_n$. The copying model minimizes it, so moving toward the copying model lowers it. Twelve points do not show that the data lie on a circle, so the objective built from them rates the chords above the circle, as the training losses show. This failure is called *memorization*.

Next we follow the same runs through training, to see what the extra steps did. I expected the extra steps to help, or at worst to change little once the training loss flattened. In two of the three draws it never flattened.

Longer training makes memorization worse. Followed through training, one panel per draw of the 12 points, the wider generator keeps lowering its training loss while its test loss rises. Between $6{,}000$ and $100{,}000$ steps the training loss keeps falling as the generator moves toward the copying model, while the test loss grows by 0.05 to 0.08. By the test loss, the generator after $6{,}000$ steps is better than the same generator after $100{,}000$, in every draw.

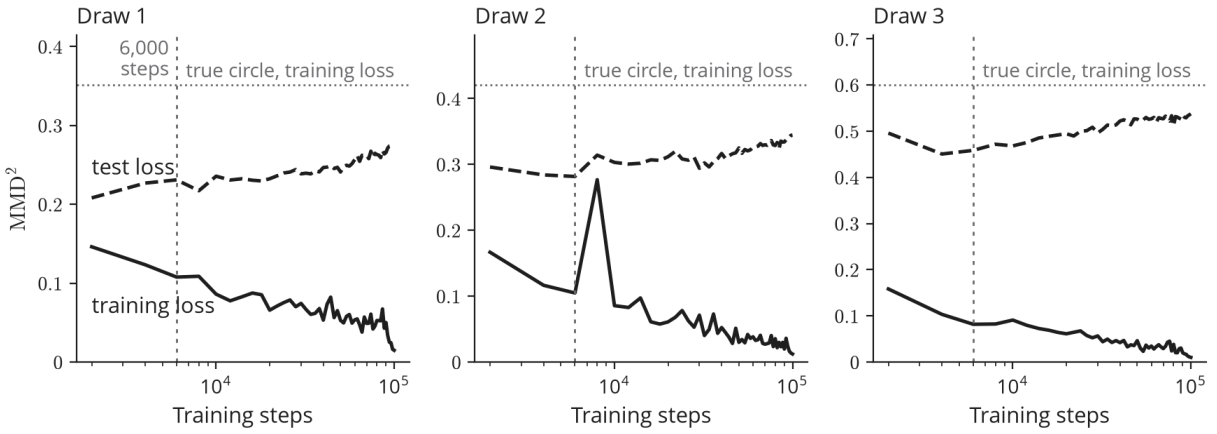


Figure 8. Training loss (solid) and test loss (dashed) of the width-1024 generator during training, one panel per draw of the 12 training points. The dotted line is the training loss of the true circle on the same points. The vertical line marks 6,000 steps. Between $6{,}000$ and $100{,}000$ steps the training loss falls from about 0.1 to about 0.01. In draws 2 and 3 it still falls by 27% and 31% between $70{,}000$ and $90{,}000$ steps. The spike in the middle panel is a brief instability of training.

More data does cure it. With more points the blurred training cloud comes close to the blurred circle, so the objective has less reason to favour the points. The wider generator was trained again on 25, 50, 100 and 200 points. With 25 points its share of copies is already close to the 1% of fresh points (1.0% to 2.5%) and falls further with more points.

The test loss needs more care. It falls at every size in every draw, but this alone does not show the cure. It stays close to *Truth*, which compares the true circle with the training points. To score the copying model on the test loss we would compare the training points with the circle, which is the same comparison, so it would also score about *Truth*. A test loss near *Truth* therefore cannot tell a generator that follows the circle from one that copies. The evidence is in the copies and in the distance to the circle.

On 200 points the samples of both widths follow the circle, but the cure is partial. They still beat *Truth* on training loss, and their test loss is still 5 to 16 times the 0.002 of fresh points, so 200 points leave a visible gap. Their copy rate is also below the 1% of fresh points, a sign that the samples sit slightly off the circle. Their median distance to it is 0.003 to 0.006, which looks small but is far larger than the radius of about 0.0002 of the discs that define a copy.

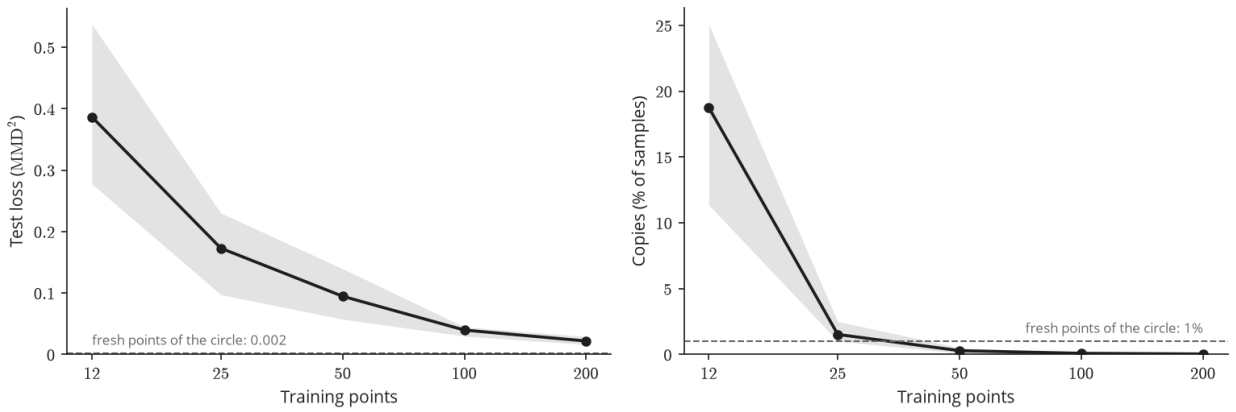


Figure 9. Test loss (left) and copies (right) of the width-1024 generator against the number of training points. Dots are the mean of the three draws, and the band spans the smallest and largest value. Dashed lines mark what fresh points of the circle score.

A reader may object that these generators have $k = d = 2$, outside the setting of Proposition 6. So we repeated the runs on 12 and 200 points with $k = 1$, where the generator has no density. The MMD needs no density, so it trains these generators all the same. Each result of the $k = 2$ runs on 12 and 200 points holds again in all three draws.

On 12 points every generator scores better than the truth on its training objective, and the wider one has the lower training loss and copies more. Its samples sit almost on the training points, with a median distance to the nearest one under 5% of the same distance for fresh points, as in the $k = 2$ runs. Stopping it at $6{,}000$ steps gave a lower test loss than training it for $100{,}000$, so longer training moved it away from the circle. On 200 points both widths copy at most 0.1% of their samples.

Width	Points	Training loss	Truth	Test loss	Copies
16	12	0.097–0.205	0.351–0.600	0.292–0.490	1.6–3.2%
1024	12	0.017–0.020	0.351–0.600	0.264–0.537	6.7–24.1%
16	200	0.012–0.015	0.016–0.025	0.008–0.017	0–0.1%
1024	200	0.005–0.006	0.016–0.025	0.012–0.026	0–0.1%

The same settings with one latent dimension ($k = 1$), with the ranges and *Truth* as in the first table.

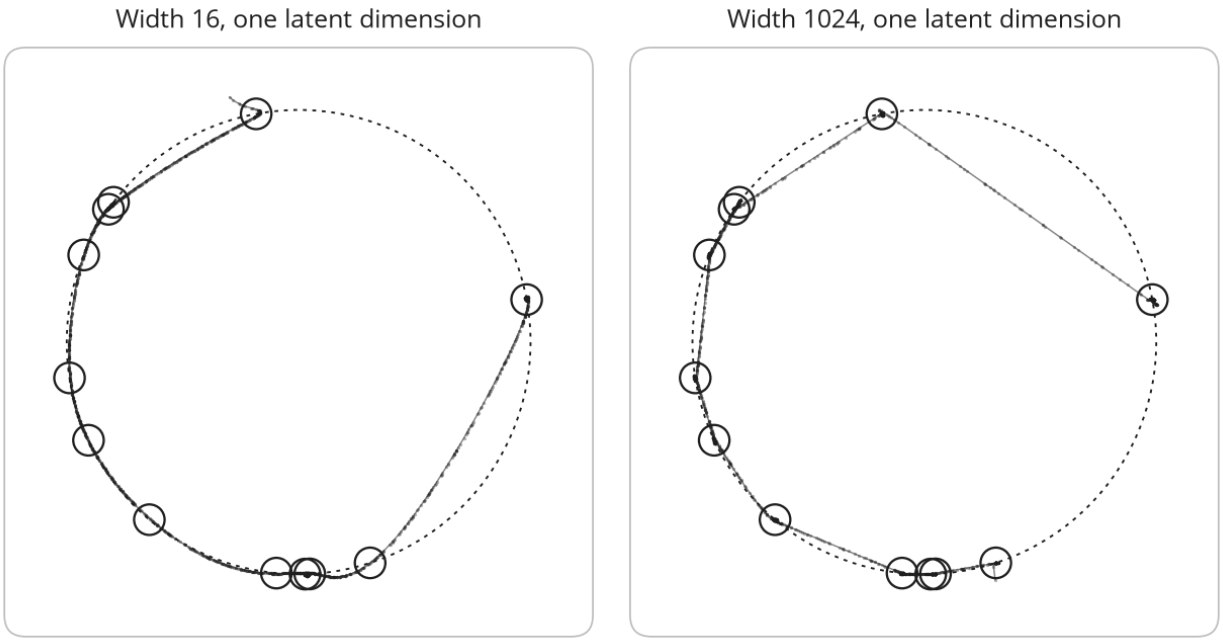


Figure 10. Generators with one latent dimension, trained on the 12 points of the second draw. The image of each generator is a curve, drawn through its samples in the order of their latent values. Open circles are the training points and the dashed line is the true circle. The wide generator runs along straight chords from one training point to the next and puts most of its samples near the points. The narrow one follows the circle more closely between the points. Both curves leave part of the circle uncovered. Each is a curve, which has zero area, so neither generator has a density.

Lab

Full results

Each setting was trained for $100{,}000$ steps with Adam, at a learning rate of 10^{-3} held for $90{,}000$ steps and then decreased to zero, and repeated with three draws of the training points. A range gives the smallest and largest of the three values. *Truth* is the training loss of $2{,}000$ points of the true circle against the same training points.

Width	Points	Training loss	Truth	Test loss	Copies
4	12	0.116–0.214	0.351–0.600	0.183–0.472	1.0–4.0%
16	12	0.054–0.106	0.351–0.600	0.256–0.518	3.6–6.6%
64	12	0.016–0.062	0.351–0.600	0.261–0.536	4.5–15.7%
256	12	0.014–0.020	0.351–0.600	0.288–0.507	9.4–23.5%
1024	12	0.010–0.015	0.351–0.600	0.277–0.538	11.4–25.1%
1024	25	0.014–0.018	0.131–0.241	0.097–0.230	1.0–2.5%
1024	50	0.010–0.012	0.063–0.152	0.057–0.139	0.1–0.5%
1024	100	0.007–0.008	0.035–0.053	0.030–0.045	0.1%
1024	200	0.004–0.007	0.016–0.025	0.016–0.029	0–0.1%
16	200	0.005–0.007	0.016–0.025	0.011–0.032	0%
4	200	0.013–0.018	0.016–0.025	0.011–0.030	0%

Width 4 has the lowest test loss of the five widths in every draw. Widths 4, 16 and 64 copy the least, in that order. The order also holds with a kernel twice narrower or twice wider (one draw).

The runs with $k = 1$ are in the second table of this section. In the first draw the width-16 generator with $k = 1$ on 12 points put 31% of its samples farther than 0.25 from every fresh point of the circle. The same run has the highest training loss of the $k = 1$ runs, 0.205. In the other $k = 1$ runs this share is at most 2.4%.

Four sources of error

The circle runs missed $\mathbb{P}^*$ in different ways. The wide generator copied 12 points and longer training made it worse, while more points fixed it. To compare such failures, and later those of other models, we need a common statement of the problem and a split of the error into its sources. We now have what we need for the statement.

Training on the circle required three choices. Training picks one network out of many, so we need a set of candidates, the model class. To pick well we need a way of saying how far the distribution of a candidate's images is from the distribution of real images. That distance usually

cannot be computed, so we also need a number computed from the training images that stands in for it. The statement below collects them with their symbols.

Problem 7 (Learning a generative model)**.** We are given n samples $X_1, \dots, X_n$ of an unknown $\mathbb{P}^*$, a model class $\mathcal{G}$ of generators and a way $D(\mathbb{P}^* \,\|\, \mathbb{P}_g)$ to measure how far the model is from the data (a distance or a divergence, Chapter 2). We look for $\widehat{g} \in \mathcal{G}$ such that $D(\mathbb{P}^* \,\|\, \mathbb{P}_{\widehat{g}})$ is small. Since $\mathbb{P}^*$ is unknown, $\widehat{g}$ is obtained by minimizing an objective $S_n(g)$ computed from the samples.

The later chapters are organised around these three choices. For each model they say which choices it makes, why its objective takes the form it does and what those choices cost. Some models pay with blurry samples (the VAE), some with unstable training (the GAN) and some with slow sampling (diffusion). These names are only labels until their own chapters. The course page lists them all.

The problem asks for a small error. To make it small we first need to know where the error of a trained model comes from. Write $R(g) = D(\mathbb{P}^* \,\|\, \mathbb{P}_g)$ for the error of a generator g, and $S(g)$ for the value the objective takes with infinitely many samples. On the circle, S_n compares the samples with the 12 training points and S compares them with the whole circle. Four generators of the class lead from the data to the trained model, which is the last of them. Each one marks something we had to give up. The best one is $g_{\mathcal{G}}$, which minimizes R over $\mathcal{G}$. It is what we would pick if we could compute R. Since we usually cannot, we settle for the generator g_S that minimizes S. Since we only have n samples, we settle again for g_{S_n}, which minimizes S_n. And since training may stop before reaching it, what it returns is $\widehat{g}$.

Generator	How it is obtained	What we gave up to get it
$g_{\mathcal{G}}$	minimizes R over $\mathcal{G}$	every distribution outside $\mathcal{G}$, and R usually cannot be computed
g_S	minimizes S	the exact distance D
g_{S_n}	minimizes S_n	infinitely many samples
$\widehat{g}$	returned by training	an exact minimization

Assuming each minimum is attained, the error of the trained generator splits into four terms,

$$R(\widehat{g}) = R(g_{\mathcal{G}}) + \big[R(g_S) - R(g_{\mathcal{G}})\big] + \big[R(g_{S_n}) - R(g_S)\big] + \big[R(\widehat{g}) - R(g_{S_n})\big].$$

Because the sum telescopes and every term is measured with the same D, the four terms add up to the total error. Each row of the table after the first contributes one term, while the first contributes the error that remains even with the best generator of the class. Of the three terms

from the later rows, only the surrogate term is sure to be non-negative, so giving something up does not always raise the error.

The term $R(g_{\mathcal{G}}) = \inf_{g \in \mathcal{G}} D(\mathbb{P}^* \,\|\, \mathbb{P}_g)$ is the *approximation error*, the distance that even the best generator of the class keeps from the data. The circle experiment shows no sign that it limits the result. On 200 points, width 4 reaches about the same test loss as the widest generator.

Next comes the *surrogate error*, $R(g_S) - R(g_{\mathcal{G}})$. Most distances D cannot be computed even with infinite data, so the objective S replaces D by something computable. Since g_S minimizes this stand-in in place of D, it can be worse than $g_{\mathcal{G}}$. Chapter 3, for example, maximizes a lower bound on the likelihood. This term is never negative, since $g_{\mathcal{G}}$ minimizes R. In the circle experiment S is the MMD itself, so the term is zero.

The *statistical error* $R(g_{S_n}) - R(g_S)$ comes from minimizing S_n in place of S, because the samples show $\mathbb{P}^*$ only through n points. The data sweep shows it. With the same architecture, the test loss falls from about 0.4 on 12 points to about 0.02 on 200. The test loss measures $R(\hat{g})$, so this drop also contains the optimization term below, which the experiment does not isolate. On 12 points stopping at $6{,}000$ steps in place of $100{,}000$ changed the test loss by only 0.05 to 0.08. That is a small part of the drop, which is therefore mostly the statistical term.

Last is the *optimization error*, $R(\hat{g}) - R(g_{S_n})$, which appears when training stops before reaching g_{S_n}.

Two of these terms can be negative, which looks odd in a list of errors, so we take this slowly. The optimization term can be negative because g_{S_n} is chosen by S_n alone, so nothing forces $\hat{g}$ to be worse than g_{S_n} when both are measured by R. On 12 points the optimization term helped. Training could return the generator at $6{,}000$ steps or at $100{,}000$. The minimizer of S_n is close to the copying model, so by the test loss the generator stopped at $6{,}000$ steps was better than the one trained to $100{,}000$. Stopping early keeps the generator from following S_n all the way to the training points.

The statistical term has less freedom. Every generator of the class has $R(g) \geq R(g_{\mathcal{G}})$, in particular g_{S_n}. Subtracting $R(g_S)$ from both sides gives $R(g_{S_n}) - R(g_S) \geq -\big[R(g_S) - R(g_{\mathcal{G}})\big]$. The right side is the surrogate term with a minus sign. When the surrogate term is zero this says the statistical term is at least zero, so it can be negative only when the surrogate term is positive, and by at most that amount. In the circle experiment the surrogate term is zero, so the statistical term is not negative.

Chapter 2 sets optimization aside and bounds the surrogate and statistical terms together by one inequality.

Check yourself

1. The empirical distribution $\mathbb{P}_n$ fits the training set perfectly. Why is it a bad answer?

   Answer

   > Sampling from it only returns training images, so it never produces a new one. The goal is to be close to $\mathbb{P}^*$, of which the training set is only a finite sample.

2. Can a generator $g : \mathbb{R}^2 \to \mathbb{R}^3$ built from linear layers and ReLU activations define a distribution with a density on $\mathbb{R}^3$?

   Answer

   > No. Such a network is Lipschitz, so by Proposition 6 its image $g(\mathbb{R}^2)$ has volume zero in $\mathbb{R}^3$. All the mass sits on that set, which no density allows.

3. MNIST has an intrinsic dimension between 7 and 13. What does that suggest about the size of the latent space?

   Answer

   > A locally Lipschitz generator needs k at least as large as the dimension of the manifold to cover it, so k should be at least about 7 to 13. This is a lower bound. It does not say how many more dimensions training needs.

4. In the circle experiment, which source of error limits the width-1024 generator trained on 12 points, and what fixed it?

   Answer

   > Statistical error. Its class can follow the circle, as the runs on 200 points show, but 12 points do not show that the data lie on a circle. Its training loss is even lower than that of the true circle on the same points. More data fixed it, while longer training made it worse.

5. On 12 points, the width-1024 generator had a better test loss after $6{,}000$ steps than after $100{,}000$. Why?

   Answer

   > The objective is minimized near the copying model, so training longer moves the generator toward the 12 points and away from the circle. Stopping early leaves some optimization error, which here keeps it closer to $\mathbb{P}^*$.

Summary

We started with a model that fits its training set perfectly and teaches us nothing, the copying model. A generator should give us the distribution $\mathbb{P}^*$ behind the samples, even though the samples are all the evidence we have about it.

To represent $\mathbb{P}^*$, a generator pushes noise through a network. One latent number was enough to produce the circle exactly. Natural images also lie near a low-dimensional manifold, so a generator whose image covers that manifold needs at least as many latent dimensions as the manifold has, far fewer than the number of pixels. This economy creates a problem. A network with fewer latent dimensions than pixels puts all its probability on a set of zero volume, so it has no density and cannot be trained by maximum likelihood unless noise is added.

Training brought a second problem. On 12 points every generator fitted the points better than the true circle did, the wider one copied more of them, and longer training pushed it further toward copying. With 200 points the same generators followed the circle. Failures like these

trace back to four sources. They are the model class, the finite sample, the objective and the optimization.

All of this leaves the number that training minimizes still to be chosen. It has to say how far $\mathbb{P}_g$ is from $\mathbb{P}^*$, it has to be computed from n samples, and it has to make sense when $\mathbb{P}_g$ has no density. Chapter 2 compares the standard choices and shows which of them still work.

Arjovsky, Martin, and Léon Bottou. 2017. “Towards Principled Methods for Training Generative Adversarial Networks.” In *International Conference on Learning Representations (ICLR)*.

Dinh, Laurent, Jascha Sohl-Dickstein, and Samy Bengio. 2017. “Density Estimation Using Real NVP.” In *International Conference on Learning Representations (ICLR)*.

Hyvärinen, Aapo. 2005. “Estimation of Non-Normalized Statistical Models by Score Matching.” *Journal of Machine Learning Research* 6: 695–709.

Oord, Aäron van den, Nal Kalchbrenner, and Koray Kavukcuoglu. 2016. “Pixel Recurrent Neural Networks.” In *International Conference on Machine Learning (ICML)*.

Pope, Phillip, Chen Zhu, Ahmed Abdelkader, Micah Goldblum, and Tom Goldstein. 2021. “The Intrinsic Dimension of Images and Its Impact on Learning.” In *International Conference on Learning Representations (ICLR)*.

Rezende, Danilo Jimenez, and Shakir Mohamed. 2015. “Variational Inference with Normalizing Flows.” In *International Conference on Machine Learning (ICML)*.

Song, Yang, and Stefano Ermon. 2019. “Generative Modeling by Estimating Gradients of the Data Distribution.” In *Advances in Neural Information Processing Systems (NeurIPS)*.

Theis, Lucas, Aäron van den Oord, and Matthias Bethge. 2016. “A Note on the Evaluation of Generative Models.” In *International Conference on Learning Representations (ICLR)*.

Vincent, Pascal. 2011. “A Connection Between Score Matching and Denoising Autoencoders.” *Neural Computation* 23 (7): 1661–74.

1. Table 1 of Pope et al. (2021). The ImageNet resolution is stated in their introduction. Each range comes from the number of neighbours used by their estimator.

❝ Cite this chapter

Bouftini, I. (2026). Learning a distribution. Chapter 1 of the course Generative Models for Images. https://ibouftini.github.io/courses/generative-models/learning-a-distribution/

```
@misc{bouftini2026learning,
  author = {Bouftini, Imade},
  title  = {Learning a distribution},
  year   = {2026},
  note   = {Chapter 1 of the course Generative Models for Images},
  url    = {https://ibouftini.github.io/courses/generative-models/learning-a-
distribution/}
}
```

Printed from https://ibouftini.github.io/courses/generative-models/learning-a-distribution/. The online version has the interactive figures and any later corrections.

© 2026 Imade Bouftini

GitHub LinkedIn